DevOps Placement Program

Week 1 – Assignment 1

# Containerize & Deploy a Multi-Tier Application on Kubernetes

*Implementation Guide & Solution*

Node.js Frontend + Java Spring Boot Backend + PostgreSQL

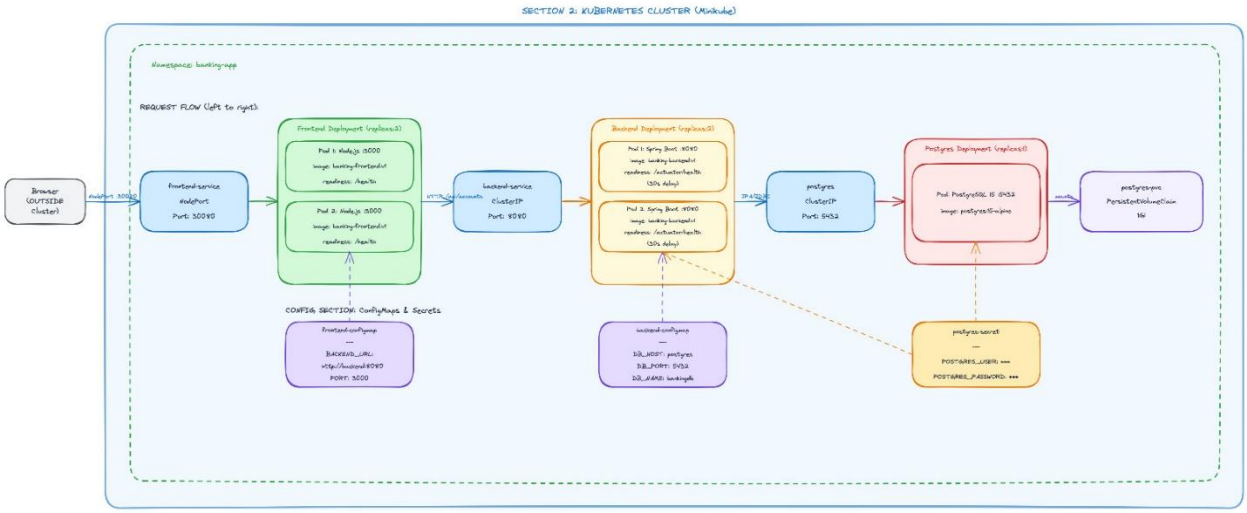
**Windows** | **macOS**

| | |
|---|---|
| **Estimated Time** | 3–4 Hours |
| **Difficulty** | Beginner–Intermediate |
| **Backend Stack** | Jav |
| **Platforms** | Windows 10/11 or macOS |

Confidential | Version 3.0 | March 2026

# Assignment Overview

In this assignment, you will take a pre-built 3-tier application (Node.js frontend + Java Spring Boot backend + PostgreSQL database) and deploy it on a local Kubernetes cluster using Minikube. The backend uses Spring Boot 3 with Java 17, Spring Data JPA for database access, and Maven as the build tool.




| | |
|---|---|
| **Frontend** | Node.js (Express + EJS) — port 3000 |
| **Backend** | Java 17 + Spring Boot 3 + Spring Data JPA — port 8080 |
| **Database** | PostgreSQL 15 — port 5432 |
| **Build Tool** | Maven (pom.xml) |
| **Where It Runs** | Minikube (local Kubernetes) |
| **Image Registry** | Docker Hub (free account) |
| **Total Steps** | 7 Steps |

| | |
|---|---|
| **Blue boxes = Windows instructions** | **Green boxes = macOS instructions** |

# Application Architecture

**Traffic Flow**

```
Browser → Frontend (Node.js :3000) → Backend API (Spring Boot :8080) → PostgreSQL (:5432)
```

| Component | Tech Stack | Port | What It Does |
|---|---|---|---|
| **Frontend** | Node.js + Express + EJS | 3000 | Serves web UI, calls backend REST API |
| **Backend** | Java 17 + Spring Boot 3 + JPA | 8080 | REST API, business logic, JPA to DB |
| **Database** | PostgreSQL 15 | 5432 | Stores all application data |

**NOTE:** The backend port is 8080 (Spring Boot default) All Kubernetes manifests and frontend config must use port 8080.

# Folder Structure

**Project Structure**

```
devops-assignment1/
├── frontend/
│   ├── src/server.js
│   ├── views/index.ejs
│   ├── package.json
│   └── Dockerfile
├── backend/
│   ├── src/main/java/com/banking/
│   │   ├── BankingApplication.java
│   │   ├── controller/AccountController.java
│   │   ├── model/Account.java
│   │   ├── repository/AccountRepository.java
│   │   └── dto/AccountRequest.java
│   ├── src/main/resources/
│   │   └── application.yaml
│   ├── pom.xml
│   └── Dockerfile
├── k8s/
│   ├── namespace.yaml
│   ├── postgres-secret.yaml
│   ├── postgres-pvc.yaml
│   ├── postgres-deployment.yaml
│   ├── postgres-service.yaml
│   ├── backend-configmap.yaml
│   ├── backend-deployment.yaml
│   ├── backend-service.yaml
│   ├── frontend-configmap.yaml
│   ├── frontend-deployment.yaml
│   └── frontend-service.yaml
```

```
├── .gitignore
└── README.md
```

## Step 1: Install Prerequisites (20 minutes)

### 1.1 — Install Docker Desktop

**Windows: Docker Desktop**

```
1. Download from: docker.com/products/docker-desktop
2. Run installer, enable 'Use WSL 2'
3. Restart PC
4. Verify: docker --version
```

**macOS: Docker Desktop**

```
1. Download from: docker.com/products/docker-desktop
2. Drag to Applications, launch
3. Wait for whale icon in menu bar
4. Verify: docker --version
```

### 1.2 — Install Minikube & kubectl

**Windows (PowerShell as Admin)**

```
winget install Kubernetes.minikube
winget install Kubernetes.kubectl

minikube version
kubectl version --client
```

**macOS (Terminal)**

```
brew install minikube
brew install kubectl

minikube version
kubectl version --client
```

### 1.3 — Install Java 17 (for local testing)

**Windows**

```
# Option 1: winget
winget install Microsoft.OpenJDK.17

# Option 2: Download from
# adoptium.net/temurin/releases

java -version
# Should show: openjdk 17.x.x
```

**macOS**

```
# Using Homebrew
brew install openjdk@17

# Add to PATH:
echo 'export PATH="/opt/homebrew/opt/
  openjdk@17/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc

java -version
```

**NOTE:** Java is only needed locally for building/testing. Inside Docker and Kubernetes, the container image includes its own JDK — your host Java version does not matter for deployment.

### 1.4 — Install Maven (for local testing)

**Windows**

```
winget install Apache.Maven
```

**macOS**

```
brew install maven

mvn -version
```

```
# Or download from: maven.apache.org
# download Download binary zip
#Extract to C:\Program Files\Maven
#Set environment variables:
#MAVEN_HOME = C:\Program Files\Maven
#Add to PATH: %MAVEN_HOME%\bin

mvn -version
```

## 1.5 — Start Minikube

**Terminal / PowerShell (same on both)**

```
minikube start --driver=docker --cpus=2 --memory=4096

kubectl get nodes
# Should show: minikube   Ready   control-plane
```

**TIP:** Spring Boot apps use more memory than Python. Allocate at least 4GB to Minikube (--memory=4096). On Windows, ensure Docker Desktop has 4GB+ allocated in Settings > Resources.

**Troubleshooting: Minikube Fails to Start**
If you encounter an error while running minikube start, ensure Docker Desktop is running before retrying.

Steps:
1. Open Docker Desktop (open -a Docker)
2. Wait until Docker is fully started (~30–60 seconds)
3. Verify with docker info — it should return clean output
4. Re-run minikube start

## Step 2: Create Project Structure (10 minutes)

**Windows (PowerShell)**

```
mkdir devops-assignment1
cd devops-assignment1
mkdir frontend\src, frontend\views
mkdir backend\src\main\java\com\banking\controller
mkdir backend\src\main\java\com\banking\model
mkdir backend\src\main\java\com\banking\repository
mkdir backend\src\main\java\com\banking\dto
mkdir backend\src\main\resources
mkdir k8s
git init
```

**macOS (Terminal)**

```
mkdir -p devops-assignment1
cd devops-assignment1
mkdir -p frontend/src frontend/views
mkdir -p
backend/src/main/java/com/banking/{controller,mode
l,repository,dto}
mkdir -p backend/src/main/resources
mkdir -p backend/src/main/resourcesmkdir -p
backend/src/main/resources
git init
```

## Step 3: Create the Backend Application (Java Spring Boot 3) (40 minutes)

This is the core backend code. Spring Boot 3 uses Java 17+, Spring Data JPA for database access, and auto-configures most of the application.

### 3.1 — Maven Configuration (pom.xml)

**File: backend/pom.xml**

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.1</version>
  </parent>

  <groupId>com.banking</groupId>
  <artifactId>banking-backend</artifactId>
  <version>1.0.0</version>
  <name>Banking Backend API</name>

  <properties>
    <java.version>17</java.version>
  </properties>

  <dependencies>
    <!-- Spring Boot Web (REST API) -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <!-- Spring Data JPA (Database) -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
```

```
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>

    <!-- PostgreSQL Driver -->
    <dependency>
      <groupId>org.postgresql</groupId>
      <artifactId>postgresql</artifactId>
      <scope>runtime</scope>
    </dependency>

    <!-- Spring Boot Actuator (health checks) -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-actuator</artifactId>
    </dependency>

    <!-- Validation -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>

    <!-- Test -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

## 3.2 — Application Configuration

**File: backend/src/main/resources/application.yaml**

```yaml
spring:
  application:
    name: banking-backend
  datasource:
    url: jdbc:postgresql://${DB_HOST:localhost}:${DB_PORT:5432}/${DB_NAME:bankingdb}
    username: ${DB_USER:postgres}
    password: ${DB_PASSWORD:postgres}
    driver-class-name: org.postgresql.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: false
    properties:
      hibernate:
        dialect: org.hibernate.dialect.PostgreSQLDialect

server:
  port: 8080

management:
  endpoints:
    web:
      exposure:
        include: health,info
  endpoint:
    health:
      show-details: always
```

**TIP:** The ${DB_HOST:localhost} syntax means: use the DB_HOST environment variable if set, otherwise default to localhost. This is how Spring Boot reads config from Kubernetes ConfigMaps and Secrets.

## 3.3 — Main Application Class

**File: backend/src/main/java/com/banking/BankingApplication.java**

```java
package com.banking;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class BankingApplication {
    public static void main(String[] args) {
        SpringApplication.run(BankingApplication.class, args);
    }
}
```

## 3.4 — Account Entity (Model)

**File: backend/src/main/java/com/banking/model/Account.java**

```
package com.banking.model;

import jakarta.persistence.*;
import java.math.BigDecimal;

@Entity
@Table(name = "accounts")
public class Account {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;

    @Column(nullable = false, unique = true, length = 100)
    private String email;

    @Column(precision = 10, scale = 2)
    private BigDecimal balance = BigDecimal.ZERO;

    // Default constructor (required by JPA)
    public Account() {}

    public Account(String name, String email, BigDecimal balance) {
        this.name = name;
        this.email = email;
        this.balance = balance;
    }

    // Getters and Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public BigDecimal getBalance() { return balance; }
    public void setBalance(BigDecimal balance) { this.balance = balance; }
}
```

## 3.5 — Repository

**File: backend/src/main/java/com/banking/repository/AccountRepository.java**

```
package com.banking.repository;

import com.banking.model.Account;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface AccountRepository extends JpaRepository<Account, Long> {
    boolean existsByEmail(String email);
}
```

## 3.6 — DTO (Request Object)

**File: backend/src/main/java/com/banking/dto/AccountRequest.java**

```
package com.banking.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import java.math.BigDecimal;

public class AccountRequest {

    @NotBlank(message = "Name is required")
    private String name;

    @NotBlank(message = "Email is required")
    @Email(message = "Invalid email format")
    private String email;

    private BigDecimal balance = BigDecimal.ZERO;

    // Getters and Setters
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public BigDecimal getBalance() { return balance; }
    public void setBalance(BigDecimal balance) { this.balance = balance; }
}
```

## 3.7 — REST Controller

**File: backend/src/main/java/com/banking/controller/AccountController.java**

```java
package com.banking.controller;

import com.banking.dto.AccountRequest;
import com.banking.model.Account;
import com.banking.repository.AccountRepository;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.server.ResponseStatusException;

import java.util.List;
import java.util.Map;

@RestController
@CrossOrigin(origins = "*")
public class AccountController {

    private final AccountRepository repo;

    public AccountController(AccountRepository repo) {
        this.repo = repo;
    }

    @GetMapping("/health")
    public Map<String, String> health() {
        return Map.of("status", "healthy", "service", "backend");
    }

    @GetMapping("/api/accounts")
    public List<Account> listAccounts() {
        return repo.findAll();
    }

    @GetMapping("/api/accounts/{id}")
    public Account getAccount(@PathVariable Long id) {
        return repo.findById(id)
            .orElseThrow(() -> new ResponseStatusException(
                HttpStatus.NOT_FOUND, "Account not found"));
    }

    @PostMapping("/api/accounts")
    public ResponseEntity<Account> createAccount(
            @Valid @RequestBody AccountRequest request) {
        if (repo.existsByEmail(request.getEmail())) {
            throw new ResponseStatusException(
                HttpStatus.BAD_REQUEST, "Email already exists");
        }
        Account account = new Account(
            request.getName(),
            request.getEmail(),
            request.getBalance()
        );
        Account saved = repo.save(account);
        return ResponseEntity.status(HttpStatus.CREATED).body(saved);
    }
}
```

**NOTE:** Spring Boot Actuator provides a /actuator/health endpoint automatically. We also create a custom /health endpoint for consistency with the frontend. The Kubernetes readiness and liveness probes will use /actuator/health.

## Step 4: Create the Frontend Application (Node.js) (15 minutes)

The frontend is the same as before, but updated to call port 8080 (Spring Boot) instead of 8000.

**File: frontend/package.json**

```
{
  "name": "banking-frontend",
  "version": "1.0.0",
  "main": "src/server.js",
  "scripts": { "start": "node src/server.js" },
  "dependencies": {
    "express": "^4.18.2",
    "ejs": "^3.1.9",
    "axios": "^1.6.2"
  }
}
```

**File: frontend/src/server.js**

```
const express = require('express');
const axios = require('axios');
const path = require('path');

const app = express();
const PORT = process.env.PORT || 3000;
const BACKEND_URL = process.env.BACKEND_URL || 'http://localhost:8080';

app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, '../views'));
app.use(express.urlencoded({ extended: true }));
app.use(express.json());

app.get('/health', (req, res) => {
  res.json({ status: 'healthy', service: 'frontend' });
});

app.get('/', async (req, res) => {
  try {
    const response = await axios.get(`${BACKEND_URL}/api/accounts`);
    res.render('index', { accounts: response.data, error: null });
  } catch (err) {
    res.render('index', { accounts: [], error: 'Backend unavailable' });
  }
});
```

```
app.post('/accounts', async (req, res) => {
  try {
    await axios.post(`${BACKEND_URL}/api/accounts`, req.body);
    res.redirect('/');
  } catch (err) { res.redirect('/'); }
});

app.listen(PORT, () => console.log(`Frontend running on port ${PORT}`));
```

**File: frontend/views/index.ejs**

```
<!DOCTYPE html>
<html><head><title>Banking App</title>
<style>
  body { font-family: Arial; max-width: 800px; margin: 50px auto; padding: 0 20px; }
  h1 { color: #1B4F72; } table { width: 100%; border-collapse: collapse; margin: 20px 0; }
  th, td { border: 1px solid #ddd; padding: 12px; text-align: left; }
  th { background: #1B4F72; color: white; }
  form { background: #f5f5f5; padding: 20px; border-radius: 8px; margin: 20px 0; }
  input { padding: 8px; margin: 5px; border: 1px solid #ddd; border-radius: 4px; }
  button { padding: 10px 20px; background: #2E86C1; color: white; border: none; border-radius:
4px; cursor: pointer; }
  .error { color: red; padding: 10px; background: #ffe0e0; border-radius: 4px; }
</style></head><body>
  <h1>Banking Application</h1>
  <% if (error) { %><p class="error"><%= error %></p><% } %>
  <h2>Create Account</h2>
  <form action="/accounts" method="POST">
    <input name="name" placeholder="Full Name" required />
    <input name="email" placeholder="Email" type="email" required />
    <input name="balance" placeholder="Balance" type="number" step="0.01" value="0" />
    <button type="submit">Create</button>
  </form>
  <h2>Accounts (<%= accounts.length %>)</h2>
  <table><tr><th>ID</th><th>Name</th><th>Email</th><th>Balance</th></tr>
    <% accounts.forEach(a => { %>
      <tr><td><%= a.id %></td><td><%= a.name %></td><td><%= a.email %></td><td>$<%=
a.balance.toFixed(2) %></td></tr>
    <% }); %>
  </table>
</body></html>
```

## Step 5: Write Multi-Stage Dockerfiles (20 minutes)

### 5.1 — Backend Dockerfile (Java Multi-Stage Build)

The Java Dockerfile uses Maven to build the JAR in stage 1, then copies only the JAR to a slim JRE image in stage 2. This reduces the final image from ~800MB to ~250MB.

**File: backend/Dockerfile**

```
# Stage 1: Build with Maven
FROM maven:3.9-eclipse-temurin-17 AS builder
WORKDIR /app
COPY pom.xml .
# Download dependencies first (cached layer)
RUN mvn dependency:go-offline -B
COPY src/ ./src/
RUN mvn package -DskipTests -B

# Stage 2: Production image (JRE only, no JDK or Maven)
FROM eclipse-temurin:17-jre
WORKDIR /app

# Copy the built JAR from stage 1
COPY --from=builder /app/target/*.jar app.jar

# Non-root user for security
RUN groupadd -r appgroup && useradd -r -g appgroup appuser
USER appuser

EXPOSE 8080

# JVM tuning for containers
ENV JAVA_OPTS="-XX:+UseContainerSupport -XX:MaxRAMPercentage=75.0"

ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS -jar app.jar"]
```

**TIP:** Key Docker optimizations: (1) mvn dependency:go-offline runs first to cache dependencies — rebuilds are fast if only code changes. (2) eclipse-temurin:17-jre-alpine uses JRE not JDK, saving ~400MB. (3) -XX:+UseContainerSupport lets the JVM respect container memory limits.

## 5.2 — Frontend Dockerfile (same as before)

**File: frontend/Dockerfile**

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install --omit=dev

FROM node:20-alpine
WORKDIR /app
COPY --from=builder /app/node_modules ./node_modules
COPY src/ ./src/
COPY views/ ./views/
COPY package.json ./
RUN addgroup -S appgroup && adduser -S -G appgroup appuser
USER appuser
EXPOSE 3000
CMD ["node", "src/server.js"]
```

### 5.3 — Build & Push Images

**Terminal / PowerShell**

```
# Build backend (takes 2-3 min first time due to Maven downloads)
docker build -t banking-backend:v1 ./backend

# Build frontend
docker build -t banking-frontend:v1 ./frontend

# Check image sizes
docker images | grep banking
# backend should be ~250MB (JRE Alpine)
# frontend should be ~180MB (Node Alpine)

# Tag and push to Docker Hub
docker login
docker tag banking-backend:v1 YOUR_USERNAME/banking-backend:v1
docker tag banking-frontend:v1 YOUR_USERNAME/banking-frontend:v1
docker tag banking-backend:v1 YOUR_USERNAME/banking-backend:latest
docker tag banking-frontend:v1 YOUR_USERNAME/banking-frontend:latest
docker push YOUR_USERNAME/banking-backend:v1
docker push YOUR_USERNAME/banking-backend:latest
docker push YOUR_USERNAME/banking-frontend:v1
docker push YOUR_USERNAME/banking-frontend:latest
```

**IMPORTANT:** The first Java Docker build takes 2–3 minutes because Maven downloads all dependencies. Subsequent builds are much faster thanks to Docker layer caching.

## Step 6: Write & Apply Kubernetes Manifests (50 minutes)

All YAML manifests and kubectl commands work the same on Windows and macOS. The key difference from the Python version: backend port is 8080 and health endpoint is /actuator/health.

### 6.1 — Namespace & PostgreSQL (same as Python version)

**File: k8s/namespace.yaml**

```
apiVersion: v1
kind: Namespace
metadata:
  name: banking-app
  labels: { app: banking, environment: dev }
```

**File: k8s/postgres-secret.yaml**

```
apiVersion: v1
kind: Secret
```

```
metadata:
  name: postgres-secret
  namespace: banking-app
type: Opaque
data:
  POSTGRES_USER: cG9zdGdyZXM=
  POSTGRES_DB: YmFua2luZ2Ri
  POSTGRES_PASSWORD: c2VjcmV0cGFzczEyMw==
```

**File: k8s/postgres-pvc.yaml**

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-pvc
  namespace: banking-app
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: 1Gi
```

**File: k8s/postgres-deployment.yaml**

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres
  namespace: banking-app
spec:
  replicas: 1
  selector:
    matchLabels: { app: postgres }
  template:
    metadata:
      labels: { app: postgres }
    spec:
      containers:
        - name: postgres
          image: postgres:15-alpine
          ports:
            - containerPort: 5432
          envFrom:
            - secretRef:
                name: postgres-secret
          volumeMounts:
            - name: storage
              mountPath: /var/lib/postgresql/data
          resources:
            requests: { cpu: 100m, memory: 256Mi }
            limits: { cpu: 500m, memory: 512Mi }
          readinessProbe:
            exec:
              command: ["pg_isready", "-U", "postgres"]
            initialDelaySeconds: 5
            periodSeconds: 10
          livenessProbe:
            exec:
              command: ["pg_isready", "-U", "postgres"]
            initialDelaySeconds: 15
            periodSeconds: 20
```

```
      volumes:
        - name: storage
          persistentVolumeClaim:
            claimName: postgres-pvc
```

**File: k8s/postgres-service.yaml**

```
apiVersion: v1
kind: Service
metadata:
  name: postgres
  namespace: banking-app
spec:
  selector: { app: postgres }
  ports:
    - port: 5432
      targetPort: 5432
  type: ClusterIP
```

## 6.2 — Backend (Spring Boot — port 8080)

Note the key differences from the Python version: port 8080, JDBC URL format, and /actuator/health for probes.

**File: k8s/backend-configmap.yaml**

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: backend-config
  namespace: banking-app
data:
  DB_HOST: postgres
  DB_PORT: "5432"
  DB_NAME: bankingdb
```

**File: k8s/backend-deployment.yaml**

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: banking-app
spec:
  replicas: 2
  selector:
    matchLabels: { app: backend }
  template:
    metadata:
      labels: { app: backend }
    spec:
      containers:
        - name: backend
          image: YOUR_USERNAME/banking-backend:v1
          ports:
            - containerPort: 8080
          envFrom:
            - configMapRef:
                name: backend-config
          env:
            - name: DB_USER
              valueFrom:
                secretKeyRef:
                  name: postgres-secret
                  key: POSTGRES_USER
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: postgres-secret
                  key: POSTGRES_PASSWORD
          resources:
            requests: { cpu: 200m, memory: 256Mi }
            limits: { cpu: 500m, memory: 512Mi }
          readinessProbe:
            httpGet:
              path: /actuator/health
              port: 8080
            initialDelaySeconds: 30
            periodSeconds: 10
          livenessProbe:
```

```
            httpGet:
              path: /actuator/health
              port: 8080
            initialDelaySeconds: 45
            periodSeconds: 15
```

**IMPORTANT:** Java/Spring Boot apps take longer to start than Python apps. Notice the higher initialDelaySeconds (30 for readiness, 45 for liveness) and higher memory limits (256Mi request, 512Mi limit). This prevents Kubernetes from killing the pod before it finishes starting.

**File: k8s/backend-service.yaml**

```
apiVersion: v1
kind: Service
metadata:
  name: backend
  namespace: banking-app
spec:
  selector: { app: backend }
  ports:
    - port: 8080
      targetPort: 8080
  type: ClusterIP
```

## 6.3 — Frontend (connects to backend:8080)

**File: k8s/frontend-configmap.yaml**

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: frontend-config
  namespace: banking-app
data:
  BACKEND_URL: http://backend:8080
  PORT: "3000"
```

**IMPORTANT:** BACKEND_URL is http://backend:8080 (not 8000). This is the Spring Boot default port. The 'backend' hostname is the Kubernetes Service name.

**File: k8s/frontend-deployment.yaml**

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  namespace: banking-app
spec:
  replicas: 2
  selector:
    matchLabels: { app: frontend }
  template:
    metadata:
      labels: { app: frontend }
    spec:
```

```
      containers:
        - name: frontend
          image: YOUR_USERNAME/banking-frontend:v1
          ports:
            - containerPort: 3000
          envFrom:
            - configMapRef:
                name: frontend-config
          resources:
            requests: { cpu: 100m, memory: 128Mi }
            limits: { cpu: 300m, memory: 256Mi }
          readinessProbe:
            httpGet: { path: /health, port: 3000 }
            initialDelaySeconds: 10
            periodSeconds: 5
          livenessProbe:
            httpGet: { path: /health, port: 3000 }
            initialDelaySeconds: 15
            periodSeconds: 10
```

**File: k8s/frontend-service.yaml**

```
apiVersion: v1
kind: Service
metadata:
  name: frontend
  namespace: banking-app
spec:
  selector: { app: frontend }
  ports:
    - port: 3000
      targetPort: 3000
      nodePort: 30080
  type: NodePort
```

## Step 7: Deploy to Minikube & Access the App (20 minutes)

**Terminal / PowerShell: Deploy in order**

```
# 1. Namespace
kubectl apply -f k8s/namespace.yaml

# 2. Secrets and ConfigMaps
kubectl apply -f k8s/postgres-secret.yaml
kubectl apply -f k8s/backend-configmap.yaml
kubectl apply -f k8s/frontend-configmap.yaml

# 3. Persistent storage
kubectl apply -f k8s/postgres-pvc.yaml

# 4. PostgreSQL
kubectl apply -f k8s/postgres-deployment.yaml
kubectl apply -f k8s/postgres-service.yaml
kubectl wait --for=condition=ready pod -l app=postgres -n banking-app --timeout=120s
```

```
# 5. Backend (wait longer for Spring Boot startup).
minikube image load banking-backend:v1
kubectl apply -f k8s/backend-deployment.yaml
kubectl apply -f k8s/backend-service.yaml
kubectl wait --for=condition=ready pod -l app=backend -n banking-app --timeout=180s

# 6. Frontend.
minikube image load banking-frontend:v1
kubectl apply -f k8s/frontend-deployment.yaml
kubectl apply -f k8s/frontend-service.yaml
```

**WARNING:** Spring Boot takes 20–40 seconds to start. The kubectl wait timeout is 180s (3 min) for backend. If pods show 0/1 Ready for 30 seconds, that is normal — wait for the JVM to finish initializing.

**Terminal / PowerShell: Verify**

```
# Check all pods (5 total: 1 postgres + 2 backend + 2 frontend)
kubectl get pods -n banking-app

# Check services
kubectl get svc -n banking-app

# Check backend health via port-forward
kubectl port-forward svc/backend 8080:8080 -n banking-app
# Open: http://localhost:8080/actuator/health
# Should show: {"status":"UP","components":{"db":{"status":"UP"},...}}
```

## Access the Application

**Windows**

```
# Option 1 (recommended):
minikube service frontend -n banking-app

# Option 2:
minikube tunnel
# Then open: http://localhost:30080

# Option 3:
kubectl port-forward svc/frontend
  3000:3000 -n banking-app
# Open: http://localhost:3000
```

**macOS**

```
# Option 1 (recommended):
minikube service frontend -n banking-app

# Option 2:
minikube tunnel
# Then open: http://localhost:30080

# Option 3:
kubectl port-forward svc/frontend \
  3000:3000 -n banking-app
# Open: http://localhost:3000
```

Create a few test accounts in the browser to verify the full flow: Frontend → Spring Boot Backend → PostgreSQL.

# Documentation, Git & Cleanup

**File: .gitignore**

```
node_modules/
target/
*.class
*.jar
.idea/
*.iml
.env
*.log
```

**Terminal / PowerShell**

```
git add .
git commit -m "Assignment 1: Java Spring Boot + Node.js on Kubernetes"
git remote add origin https://github.com/YOUR_USERNAME/devops-assignment1.git
git branch -M main
git push -u origin main
```

**Cleanup**

```
kubectl delete namespace banking-app
minikube stop
minikube delete    # when fully done
```

# Final Verification Checklist

| Verification Checklist |
|---|
| ✓ Minikube cluster started with Docker driver |
| ✓ Backend Docker image built with multi-stage Maven build |
| ✓ Frontend Docker image built with multi-stage Node build |
| ✓ Images pushed to Docker Hub with v1 and latest tags |
| ✓ All 5 pods Running (1 postgres + 2 backend + 2 frontend) |
| ✓ PVC is Bound (database data persists across restarts) |
| ✓ Backend /actuator/health shows UP with database component healthy |
| ✓ Frontend /health returns {"status": "healthy"} |

| | |
|---|---|
| ✓ | Frontend accessible in browser, can create accounts |
| ✓ | All pods have resource limits, readiness probes, and liveness probes |
| ✓ | Backend uses higher memory (256Mi-512Mi) and longer startup delays (30-45s) |
| ✓ | ConfigMaps for non-sensitive config, Secrets for DB credentials |
| ✓ | GitHub repo has all code, manifests, Dockerfiles, pom.xml, and README |

# Common Issues & Troubleshooting

| Problem | Solution |
|---|---|
| **Maven build fails in Docker** | Check pom.xml is in the backend/ root directory. Ensure you have internet access (Maven downloads dependencies). First build takes 2-3 minutes. |
| **Backend pod OOMKilled** | Spring Boot needs more memory than Python. Increase memory limits to 512Mi or 768Mi in backend-deployment.yaml. Also ensure Minikube has 4GB+ allocated. |
| **Backend pod CrashLoopBackOff** | Check logs: kubectl logs -l app=backend -n banking-app. Common causes: (1) PostgreSQL not ready yet, (2) wrong DB credentials in Secret, (3) insufficient memory. |
| **Backend takes 30+ seconds to start** | This is NORMAL for Spring Boot. The readiness probe has initialDelaySeconds: 30 for this reason. Wait for the pod to show 1/1 Ready. |
| **Frontend shows Backend unavailable** | Check BACKEND_URL in frontend ConfigMap is http://backend:8080 (not 8000). Spring Boot uses 8080 by default. |
| **Minikube out of memory** | Spring Boot pods use more RAM. Start Minikube with: minikube start --memory=4096. Or reduce backend replicas to 1. |
| **/actuator/health shows DOWN** | The database connection failed. Check: (1) postgres pod is Running, (2) postgres service exists, (3) DB credentials in Secret match application.yaml placeholders. |

**Assignment 1 Complete!** Next: Week 1 – Assignment 2 — CI/CD Pipeline with GitHub Actions